PLAN DE PRUEBAS

TuHospedaje

Plan y Reporte de Pruebas de Software — Sprint 4

TuHospedaje — Reservas, Historial, WhatsApp, Email, Panel de Administración, Autenticación Segura y Cancelación de Reservas

152 de 152 verificados
ESCENARIOS FUNCIONALES

422 de 422
BACKEND EN CI

416 de 416 (53 archivos)
FRONTEND EN CI

Equipo de Desarrollo · Junio-Julio 2026

Foco del Incremento: Flujo completo de reservas, historial personal, WhatsApp y email (Inc. 1) · Tablas uniformes y dashboard administrativo (Inc. 2) · Refactor de arquitectura — búsqueda paginada, i18n y route guards (Inc. 3) · Autenticación segura, cancelación de reservas y confiabilidad de frontend (Inc. 4) **Enfoque de Testing:** Tests unitarios (JUnit 5 + Mockito / Vitest + React Testing Library), Tests de integración (MockMvc + Testcontainers), Verificación de UI Manual, Suite E2E con Playwright. Los Incrementos 3 y 4 se desarrollaron bajo TDD estricto en el backend (test en rojo antes que la implementación en cada tarea).

1. Matriz Detallada de Casos de Prueba (Test Cases)

TC-30: Seleccionar Fecha y Autenticación (US #30)

- **Historias de Usuario Asociadas:** US #30 (Seleccionar fecha de check-in/check-out)
- **Precondiciones:** Servidor backend activo. Al menos un alojamiento con reservas CONFIRMED existentes en BD.
- **Tipos de Verificación:** UI Manual, Test de Integración Automatizado (JUnit 5 + MockMvc + Testcontainers).

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	Click "Reservar" en `ProductDetail` sin login	Aparece link a `/login` en lugar del botón de reserva	✔ Pasa
2	Acceder a `/booking/:id` sin sesión activa	`RequireAuth` redirige a `/login` con mensaje de autenticación	✔ Pasa
3	Mensaje en `/login` incluye mención a registro	Texto "Necesitás iniciar sesión para continuar. Si no tenés cuenta, podés registrarte." visible	✔ Pasa
4	Login exitoso desde el flujo de redirección	Vuelve automáticamente a `/booking/:id` (state `from` preservado)	✔ Pasa
5	DatePicker muestra fechas ocupadas deshabilitadas al cargar	`GET /api/lodgings/{id}/availability` invocado al montar. Rangos CONFIRMED bloqueados visualmente	✔ Pasa
6	Intento de seleccionar fecha dentro de rango ocupado	Fecha no seleccionable. Cursor indica estado deshabilitado	✔ Pasa

TC-31: Visualizar Detalles de Reserva (US #31)

- **Historias de Usuario Asociadas:** US #31 (Visualizar detalles del alojamiento al reservar)
- **Precondiciones:** Usuario autenticado con JWT válido. Alojamiento existente con imagen, descripción y features.
- **Tipos de Verificación:** UI Manual, Test Unitario de Componente (frontend).

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	Navegar a `/booking/:id` como usuario autenticado	Columna izquierda muestra nombre y ciudad del alojamiento	✔ Pasa
2	Verificar precio por noche en columna izquierda	Precio visible con formato correcto	✔ Pasa
3	Verificar imagen del alojamiento	Imagen cargada. Placeholder visible si falla la carga	✔ Pasa
4	Verificar descripción del alojamiento	Texto de descripción visible	✔ Pasa
5	Verificar lista de features con íconos Lucide	Features visibles con íconos SVG	✔ Pasa

6	Datos del usuario pre-cargados (nombre, apellido, email)	Campos read-only con los datos del usuario autenticado	✔ Pasa
7	Campo teléfono en formulario	Input activo y editable	✔ Pasa
8	DatePickers con fechas pre-cargadas desde `ProductDetail`	Fechas seleccionadas previamente visibles en los pickers	✔ Pasa
9	Cambiar fechas en el formulario	Total estimado se recalcula dinámicamente (`días × pricePerNight`)	✔ Pasa

Cobertura automatizada (frontend): `BookingPage.test.jsx` — pasos 3 y 5 (imagen y features del alojamiento en el resumen) no tenían assertion automatizada hasta este pase de auditoría; ahora cubiertos con casos que verifican `src` / `alt` de la imagen real y los nombres de las features reales, además del caso "sin imagen/features" (commit `2bced9d`).

TC-32: Realizar Reserva (US #32)

- **Historias de Usuario Asociadas:** US #32 (Realizar reserva y recibir confirmación)
- **Precondiciones:** Usuario autenticado con JWT válido. Alojamiento existente sin solapamiento de fechas en el rango elegido.
- **Tipos de Verificación:** API Rest, Test de Integración Automatizado (backend), Test Unitario de Componente (frontend), UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`POST /api/reservations` con datos válidos y token JWT	HTTP 201 Created. Body con `id` , fechas, `totalPrice` , `status: CONFIRMED`	✔ Pasa
2	`POST /api/reservations` con fechas solapadas a una reserva CONFIRMED	HTTP 409 Conflict	✔ Pasa
3	`POST /api/reservations` sin token JWT	HTTP 401 Unauthorized o 403 Forbidden	✔ Pasa
4	Confirmar reserva exitosa desde `BookingPage`	Navegación a `/booking/confirmation` con datos de la reserva	✔ Pasa
5	`/booking/confirmation` muestra nombre del alojamiento, fechas, huésped y total	Todos los datos correctos visibles	✔ Pasa
6	`/booking/confirmation` muestra mensaje de email enviado	Texto "Te enviamos un email de confirmación" visible	✔ Pasa
7	Link "Ver mis reservas" en la pantalla de confirmación	Navega a `/my-reservations`	✔ Pasa
8	Acceder a `/booking/confirmation` directamente sin reserva previa	Redirige a home (`/`)	✔ Pasa
9	Error al reservar (ej: 409 Conflict)	Mensaje de error específico visible en el formulario	✔ Pasa

Cobertura automatizada (frontend): `BookingConfirmation.test.jsx` — 4 tests unitarios: detalles de reserva (nombre, fechas formateadas `dd/mm/aaaa` , huésped, total), nota de email, links a `/my-reservations` y `/` , redirección a `/` sin state.

TC-33: Acceder al Historial de Reservas (US #33)

- **Historias de Usuario Asociadas:** US #33 (Acceder al historial personal de reservas)
- **Precondiciones:** Usuario autenticado con al menos una reserva creada. Servidor backend activo.
- **Tipos de Verificación:** API Rest, Test de Integración Automatizado (backend), Test Unitario de Componente (frontend), UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`GET /api/reservations/my` con token JWT válido	HTTP 200. Array con las reservas del usuario autenticado	✔ Pasa
2	`GET /api/reservations/my` sin token JWT	HTTP 401 Unauthorized o 403 Forbidden	✔ Pasa
3	Acceder a `/my-reservations` sin sesión activa	`RequireAuth` redirige a `/login`	✔ Pasa
4	Lista muestra nombre del alojamiento, ciudad, fechas y estado	Todos los campos visibles y correctos	✔ Pasa
5	Lista muestra datos de contacto y total de la reserva	Teléfono, email y precio total visibles	✔ Pasa
6	Lista ordenada por `checkIn` descendente	La reserva más reciente aparece primera	✔ Pasa
7	Link "Mis reservas" en el `Header` para usuario autenticado	Visible para usuarios con sesión activa. Navega a `/my-reservations`	✔ Pasa
8	Link "Mis reservas" NO visible para usuarios anónimos	No renderiza en header cuando no hay sesión	✔ Pasa
9	`GET /api/reservations/my` como usuario A — no retorna reservas de usuario B	Solo se devuelven las reservas del usuario autenticado (aislamiento de datos)	✔ Pasa

Cobertura automatizada (backend): `ReservationServiceImplTest` — `getMyReservations_returnsReservationsMappedToResponse`, `getMyReservations_returnsEmptyListWhenNoReservationsExist`. `ReservationControllerIntegrationTest` — `shouldReturnUserReservationsOrderedByCheckInDesc`, `shouldReturnOnlyAuthenticatedUserOwnReservations`.

Cobertura automatizada (frontend): `MyReservationsPage.test.jsx` — lista con noches calculadas, singular/plural, estado vacío con CTA, error de fetch. `Header.test.jsx` — "Mis reservas" solo para autenticados; login/register para anónimos; logout visible/oculto según sesión.

TC-34: Botón de WhatsApp (US #34)

- **Historias de Usuario Asociadas:** US #34 (Iniciar conversación de WhatsApp con mensaje pre-cargado)
- **Precondiciones:** Variable de entorno `VITE_WHATSAPP_NUMBER` definida en `.env`. Servidor frontend activo.
- **Tipos de Verificación:** UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO

1	Cargar cualquier página pública con `VITE_WHATSAPP_NUMBER` definida	Botón flotante de WhatsApp visible en esquina inferior derecha	✔ Pasa
2	`VITE_WHATSAPP_NUMBER` no definida en entorno	Componente `WhatsAppButton` no renderiza en el DOM	✔ Pasa
3	Click en botón de WhatsApp	Abre `wa.me/{número}` con mensaje pre-cargado en nueva pestaña	✔ Pasa
4	Verificar acceso sin login	Botón visible para usuarios anónimos. No requiere autenticación	✔ Pasa
5	Hacer scroll vertical en la página	Botón permanece en posición fija (`bottom: 24px; right: 24px`)	✔ Pasa

Cobertura automatizada (frontend): `WhatsAppButton.test.jsx` — no renderiza con env vacía; URL `wa.me` correcta con número configurado; atributos `target="_blank"` y `rel="noreferrer"` ; posicionamiento fijo; sin requisito de autenticación.

TC-35: Email de Confirmación (US #35)

- **Historias de Usuario Asociadas:** US #35 (Recibir email de confirmación de reserva)
- **Precondiciones:** Servidor backend activo. Para SMTP: `MAIL_SMTP_ENABLED=true` y credenciales Mailtrap configuradas en variables de entorno.
- **Tipos de Verificación:** API Rest, Test de Integración Automatizado, Verificación de inbox (Mailtrap).

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`POST /api/reservations` con datos válidos (modo default)	`EmailService.sendReservationConfirmation()` invocado por el servicio	✔ Pasa
2	Revisar consola del servidor con `ConsoleEmailServiceImpl` activo	Log con datos de la reserva (nombre alojamiento, fechas, huésped, total) visible en stdout	✔ Pasa
3	`POST /api/reservations` con `MAIL_SMTP_ENABLED=true` y credenciales Mailtrap	Email recibido en inbox de Mailtrap	✔ Pasa
4	Verificar contenido del email recibido en Mailtrap	Contiene: nombre del alojamiento, checkIn, checkOut, nombre del huésped, total, estado CONFIRMED	✔ Pasa
5	Verificar destinatario del email	Email enviado a la dirección `guestEmail` de la reserva	✔ Pasa

Cobertura automatizada (backend) — corrección de esta auditoría: hasta este pase, `ReservationServiceImpl.createReservation()` y `AuthServiceImpl.register()` invocaban el envío de email dentro de la misma transacción `@Transactional` que persiste la reserva/el usuario, y `SmtpEmailServiceImpl.send()` solo capturaba `MessagingException` — una falla real de SMTP (`MailException` , no checked) podía revertir una reserva o un registro exitosos. Corregido: ambos métodos ahora difieren el envío a `TransactionSynchronization#afterCommit` (mismo patrón que ya usaba `cancelReservation()`), con la excepción atrapada y logueada, nunca propagada; `send()` ahora también atrapa `MailException` . Test: `AuthServiceImplTest.shouldSendWelcomeEmailOnRegister` (mismo patrón aplicado a US #19, email de bienvenida). Commits `2bced9d` y `091df56` .

TC-36: Edición de Alojamiento en Panel Admin (Mejora complementaria)

- **Historias de Usuario Asociadas:** Mejora complementaria — edición de alojamientos en admin
- **Precondiciones:** Usuario con rol ADMIN autenticado. Al menos un alojamiento existente en BD.
- **Tipos de Verificación:** API Rest, UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	Tabla de alojamientos en panel admin	Botón "Editar" visible en cada fila	✔ Pasa
2	Click en "Editar" de un alojamiento	`LodgingFormModal` se abre con título "Editar alojamiento" y campos pre-llenados	✔ Pasa
3	Verificar datos pre-llenados	Nombre, ciudad, descripción, precio, capacidad y demás campos con los valores actuales	✔ Pasa
4	Modificar nombre y hacer click en "Guardar"	`PUT /api/lodgings/{id}` → HTTP 200. Tabla actualizada con nuevo nombre	✔ Pasa
5	`PUT /api/lodgings/{id}` sin token ADMIN	HTTP 403 Forbidden	✔ Pasa
6	Abrir modal de edición y cerrar sin modificar nada	Modal cierra sin mostrar `ConfirmDialog`	✔ Pasa
7	Modificar un campo y click en "Cancelar"	`ConfirmDialog` aparece preguntando si descartar cambios	✔ Pasa
8	Confirmar descarte en `ConfirmDialog`	Modal cierra. Alojamiento no modificado en BD	✔ Pasa

TC-37: Suite E2E con Playwright

- **Historias de Usuario Asociadas:** Transversal — cobertura end-to-end de los flujos críticos de la aplicación, incorporada por iniciativa del equipo fuera del alcance original del sprint y ampliada progresivamente en los Incrementos 2 a 4
- **Precondiciones:** Backend activo en `:8080` y frontend en `:5173`. Credenciales de usuario de prueba configuradas en `e2e/.env`.
- **Tipos de Verificación:** Test E2E Automatizado (Playwright, Chromium + Firefox), con Page Object Model y fixtures de autenticación.

SPEC	ESCENARIOS	COBERTURA	ESTADO
`smoke.spec.js`	3	Carga de home con formulario de búsqueda, página de login y página de registro	✔ Pasa
`auth.spec.js`	5	Login válido (nombre en header), login con contraseña incorrecta (error visible), logout con redirección a home, token expirado en endpoint protegido, registro exitoso (Inc. 4 — actualizado para el flujo cookie + CSRF)	✔ Pasa
`search.spec.js`	2	Búsqueda por ciudad navega a `/search`, página de resultados renderiza encabezado	✔ Pasa
`reservations.spec.js`	2	Historial de reservas carga para usuario autenticado, usuario anónimo es redirigido por `RequireAuth`	✔ Pasa

`admin-smoke.spec.js`	2	Login ADMIN carga `/admin` con todas las pestañas de navegación visibles; cada pestaña de entidad es alcanzable	✔ Pasa
`admin-categories.spec.js`	4	CRUD de categorías desde el panel: alta, edición, baja vía `ConfirmDialog`, validación de nombre vacío	✔ Pasa
`admin-features.spec.js`	4	CRUD de características: alta, edición, baja vía `window.confirm`, validación de nombre vacío	✔ Pasa
`admin-policies.spec.js`	4	CRUD de políticas: alta, edición, baja vía `window.confirm`, validación de nombre vacío	✔ Pasa
`admin-lodgings.spec.js`	5	CRUD de alojamientos (sin imagen): alta, edición, baja vía `ConfirmDialog`, validación de campos requeridos y de formato de email	✔ Pasa
`admin-reservations.spec.js`	2	Sección de reservas del panel carga sin error; tabla visible cuando hay datos	✔ Pasa
`admin-users.spec.js`	2	Tabla de usuarios carga con al menos una fila	✔ Pasa
`verify-cookie-auth.spec.js`	4	**(Inc. 4, nuevo)** Login setea `ACCESS_TOKEN` sin exponer el token en el cuerpo; la sesión sobrevive a un reload vía `/me`; logout limpia la cookie; una mutación sin header CSRF es rechazada con 403	✔ Pasa

Resumen histórico de ejecución: cada escenario se ejecutó en Chromium y Firefox: 45 escenarios × 2 navegadores = **90 ejecuciones** — 44 aprobadas y 46 omitidas en CI porque ese entorno no tenía credenciales de usuario de prueba (una condición del entorno, no un fallo; ver Sección 2).

TC-38: Tablas Administrativas Uniformes (US #36, Incremento 2)

- **Historias de Usuario Asociadas:** US #36 (tablas uniformes con ordenamiento y paginación local en el alcance histórico del Incremento 2)
- **Precondiciones:** Usuario con rol ADMIN autenticado. Registros suficientes en Categorías, Características, Políticas, Usuarios y Alojamientos para paginar.
- **Tipos de Verificación:** Test Unitario de Componente (frontend), UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	Click en el encabezado de una columna ordenable	Lista se reordena; indicador ▲/▼ visible en la columna activa	✔ Pasa
2	Click nuevamente en el mismo encabezado	Se invierte la dirección de orden	✔ Pasa
3	Navegar a la página siguiente con `Pagination`	Muestra el siguiente subconjunto de registros, paginado en el cliente	✔ Pasa
4	Botones de paginación en el límite (primera/última página)	Se deshabilitan correctamente, sin navegación fuera de rango	✔ Pasa
5	Repetir en `AdminLodgings` después de la migración histórica a una petición plana `GET /api/lodgings`	El ordenamiento y la paginación coinciden con los de las demás entidades administrativas en ese incremento	✔ Pasa

Cobertura automatizada (frontend): `useTableData.test.js` (ordenamiento, paginación, filtrado), `Pagination.test.jsx` (deshabilitado en límites), `AdminCategories/AdminFeatures/AdminPolicies/AdminUsers/AdminLodgings.test.jsx`.

TC-39: Dashboard — Reservas Recientes (US #37, Incremento 2)

- **Historias de Usuario Asociadas:** US #37 (Estadísticas de reservas y reservas recientes)
- **Precondiciones:** Usuario con rol ADMIN autenticado. Al menos 4 reservas existentes en el sistema.
- **Tipos de Verificación:** API Rest, Test de Integración Automatizado (backend), Test Unitario de Componente (frontend), UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`GET /api/reservations` con token ADMIN	HTTP 200. Listado completo ordenado por `id DESC`	✔ Pasa
2	`GET /api/reservations` con token de usuario no-ADMIN	HTTP 403 Forbidden	✔ Pasa
3	`GET /api/reservations` sin token	HTTP 401 Unauthorized	✔ Pasa
4	Cargar `AdminDashboard` como ADMIN	Tarjeta de estadística "Reservas" visible con el total	✔ Pasa
5	Sección "Últimas reservas" del Dashboard	Muestra las 4 transacciones más recientes, ordenadas por `id DESC`	✔ Pasa

Cobertura automatizada: `ReservationControllerIntegrationTest` (RBAC 401/403/200, orden `id DESC`), `AdminDashboard.test.jsx`, `AdminReservations.test.jsx`.

TC-40: Búsqueda Multi-Categoría Paginada en Servidor (US #38, Incremento 3)

- **Historias de Usuario Asociadas:** US #38 (Búsqueda por múltiples categorías con paginación server-side)
- **Precondiciones:** Al menos 2 alojamientos en categorías distintas y suficientes registros para superar una página (`size` default 9).
- **Tipos de Verificación:** API Rest, Test de Integración Automatizado (backend, Testcontainers), Test Unitario de Componente (frontend), UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`GET /api/lodgings/search` sin parámetros de paginación	Retorna `{lodgings, currentPage: 0, totalItems, totalPages}` con máximo 9 ítems	✔ Pasa
2	`GET /api/lodgings/search?categories=1,2&page=1&size=5`	Solo alojamientos de categorías 1 o 2; página 1 con máximo 5 ítems	✔ Pasa
3	`GET /api/lodgings/search?page=-1`	HTTP 400 Bad Request	✔ Pasa
4	`GET /api/lodgings/search?size=0`	HTTP 400 Bad Request	✔ Pasa
5	`GET /api/lodgings/search?page=999` (fuera de rango)	HTTP 200 con `lodgings` vacío y `currentPage: 999`	✔ Pasa
6	Seleccionar 2+ categorías en el sidebar de `SearchResults`	Un único fetch al backend con `categories` repetido; sin filtrado en memoria	✔ Pasa
7	Click en control de paginación tras aplicar filtros	Refetch real al servidor con los mismos filtros aplicados, no un slice local	✔ Pasa

Cobertura automatizada (backend): `LodgingServiceImplTest` (filtro `IN`, paginación, defaults, fuera de rango), `LodgingControllerIntegrationTest` (7 escenarios de contrato, validación `page` / `size`).

Cobertura automatizada (frontend): `SearchResults.test.jsx` (filtrado server-side, respuesta paginada, refetch en paginación), `lodgingService.test.js`.

TC-41: Mensajes de Error Localizados (US #39, Incremento 3)

- **Historias de Usuario Asociadas:** US #39 (Errores en español o inglés según `Accept-Language`)
- **Precondiciones:** Servidor backend activo. Al menos un escenario que dispare `ResourceNotFoundException` (ej. `GET /api/reservations/{id}` inexistente) y uno de validación (`page` / `size` inválidos en `/search`).
- **Tipos de Verificación:** API Rest, Test de Integración Automatizado (backend).

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`GET /api/lodgings/search?page=-1` con `Accept-Language: es`	400, `{"error":"El índice de página no debe ser negativo."}`	✔ Pasa
2	`GET /api/lodgings/search?page=-1` sin header (o `en`)	400, `{"error":"Page index must not be negative."}`	✔ Pasa
3	`GET /api/lodgings/search?size=0` con `Accept-Language: es`	400, `{"error":"El tamaño debe ser mayor a cero."}`	✔ Pasa
4	Endpoint que dispare un `IllegalArgumentException` preexistente (ej. registro con email duplicado) con `Accept-Language: en`	Mensaje se mantiene en español (13 sitios preexistentes no localizados — comportamiento documentado, no una regresión)	✔ Pasa
5	Cualquier excepción manejada, verificar logs del servidor	Logs siempre en inglés, independientemente de `Accept-Language` del cliente	✔ Pasa

Cobertura automatizada: `GlobalExceptionHandlerTest` (4 handlers en scope, ambos idiomas), `LodgingControllerIntegrationTest`, `ReservationControllerIntegrationTest` (caso real de `ResourceNotFoundException`).

TC-42: Ruta Administrativa con Redirección a `/unauthorized` (US #40, Incremento 3)

- **Historias de Usuario Asociadas:** US #40 (Acceso uniforme a rutas administrativas)
- **Precondiciones:** Un usuario autenticado con rol `USER` (no ADMIN) y un usuario con rol `ADMIN`.
- **Tipos de Verificación:** Test Unitario de Componente (frontend), UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	Usuario no autenticado navega a `/admin`	`RequireAdmin` redirige a `/login`	✔ Pasa
2	Usuario autenticado con rol `USER` navega a `/admin`	Redirige a `/unauthorized` (antes: `/`, sin explicación)	✔ Pasa
3	Página `/unauthorized`	Muestra mensaje claro y link de vuelta a `/`	✔ Pasa
4	Usuario autenticado con rol `ADMIN` navega a `/admin`	Renderiza la vista de administración normalmente	✔ Pasa

Cobertura automatizada: `RequireAdmin.test.jsx` (3 escenarios de guard), `Unauthorized.test.jsx`.

TC-43: Autenticación por Cookie HttpOnly con Protección CSRF (US #41, Incremento 4)

- **Historias de Usuario Asociadas:** US #41 (Iniciar sesión sin exponer el JWT a JavaScript, protegido contra CSRF)
- **Precondiciones:** Servidor backend activo. Usuario registrado.
- **Tipos de Verificación:** API Rest, Test de Integración Automatizado (backend, MockMvc con la cadena real de Spring Security), Test Unitario de Componente (frontend), Verificación manual con `curl`.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`POST /api/auth/login` con credenciales válidas	HTTP 200. `Set-Cookie: ACCESS_TOKEN` `HttpOnly`. Cuerpo sin el JWT	✔ Pasa
2	`GET /api/auth/csrf` sin cookie de sesión	HTTP 401. Sin `Set-Cookie: XSRF-TOKEN`	✔ Pasa
3	`GET /api/auth/csrf` con `ACCESS_TOKEN` válida	HTTP 204. `Set-Cookie: XSRF-TOKEN` legible por JavaScript (no `HttpOnly`)	✔ Pasa
4	`POST /api/auth/logout` con cookie CSRF y header `X-XSRF-TOKEN` coincidentes	HTTP 204. `ACCESS_TOKEN` limpiada	✔ Pasa
5	`POST /api/auth/logout` sin header `X-XSRF-TOKEN`, o con un valor no coincidente	HTTP 403. Sesión no afectada	✔ Pasa
6	Login exitoso repetido en distintos requests con una cookia CSRF preexistente	La cookie CSRF nunca rota en requests posteriores no relacionados con login/registro (verificado con `curl`, 6/6 reusos estables)	✔ Pasa
7	Login/registro con una cookie CSRF preexistente (posible fixation)	La respuesta de login/registro rota el token a uno nuevo, inmediatamente usable para un logout en la misma verificación	✔ Pasa

Cobertura automatizada (backend): `AuthCsrfLifecycleIntegrationTest` (7 tests — bootstrap, rotación atómica en login/registro, no-rotación en requests no relacionados, rechazo de token faltante/mismatcheado), `AuthControllerIntegrationTest`, `AuthCookieFactoryTest`, `JwtAuthenticationFilterIntegrationTest`. **Cobertura automatizada (frontend):** `AuthContext.test.jsx`, `AuthContextCsrfRace.test.jsx` (secuenciación bootstrap-antes-de-publicar-estado, condiciones de carrera), `HeaderCsrf.test.jsx`, `api.csrf.test.js`. **Cobertura E2E:** `auth.spec.js` (actualizado), `verify-cookie-auth.spec.js` (nuevo).

Nota histórica — sesiones renovables: en el corte original de este plan, la infraestructura de persistencia, rotación y detección de replay estaba cubierta de forma aislada y no estaba conectada a ningún endpoint (`app.session.refresh.enabled=false`). La integración actual y sus endpoints están documentados en la Sección 2.1.

TC-44: Cancelación de Reserva Propia (US #42, Incremento 4)

- **Historias de Usuario Asociadas:** US #42 (Cancelar una reserva propia antes del check-in)
- **Precondiciones:** Usuario autenticado con al menos una reserva `CONFIRMED` propia, con check-in futuro.

- **Tipos de Verificación:** API Rest, Test de Integración Automatizado (backend, MockMvc con CSRF real), Test Unitario (backend, concurrencia), Test Unitario de Componente (frontend), UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`PATCH /api/reservations/{id}/cancel` sobre una reserva propia `CONFIRMED` con check-in futuro, con CSRF válido	HTTP 200. `status: CANCELLED` en la respuesta	✔ Pasa
2	Repetir la misma cancelación sobre la reserva ya `CANCELLED`	HTTP 200 idempotente. No se reenvía el email de cancelación	✔ Pasa
3	`PATCH .../cancel` sobre una reserva ajena, o un `id` inexistente	HTTP 404 (idéntico en ambos casos — sin filtrar existencia a un no-propietario)	✔ Pasa
4	`PATCH .../cancel` sobre una reserva con check-in igual o anterior a la fecha de negocio actual	HTTP 400. Estado no modificado	✔ Pasa
5	`PATCH .../cancel` sin CSRF válido	HTTP 403	✔ Pasa
6	Dos requests de cancelación concurrentes sobre la misma reserva	Una única transición de estado y un único email enviado (lock pesimista)	✔ Pasa
7	Botón "Cancelar" en `MyReservationsPage` para una reserva `CONFIRMED` con check-in futuro	Botón visible; requiere confirmación antes de enviar	✔ Pasa
8	Botón "Cancelar" para una reserva ya `CANCELLED`, o con check-in pasado	Botón no ofrecido	✔ Pasa
9	Doble click rápido en "Cancelar"	Solo se envía una solicitud; botón deshabilitado mientras está en curso ("Cancelando...")	✔ Pasa
10	Cancelación falla en el servidor (ej. red caída)	Error visible en la fila afectada; la fila permanece usable, sin bloquear el resto de la lista	✔ Pasa

Cobertura automatizada (backend): `ReservationCancellationServiceTest` (límite de corte al segundo exacto de la fecha de negocio, no-propietario/inexistente, idempotencia, fallo de email no revierte la cancelación), `ReservationCancellationConcurrencyTest`, 6 casos nuevos en `ReservationControllerIntegrationTest`. **Cobertura automatizada (frontend):** 4 casos nuevos en `MyReservationsPage.test.jsx`.

TC-45: Carga Diferida de Rutas y Metadata en Español (US #43, Incremento 4)

- **Historias de Usuario Asociadas:** US #43 (Que un fallo de carga de una página no deje la aplicación en blanco)
- **Precondiciones:** Aplicación frontend construida y servida (build de producción o dev server).
- **Tipos de Verificación:** Test Unitario de Componente (frontend, con fake timers), UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	Navegar a cualquier ruta en una conexión rápida	No se muestra spinner de carga (resuelve antes de 150ms)	✔ Pasa
2	Simular una carga de chunk que tarda más de 150ms	Spinner accesible (`role="status"`) visible con el texto correcto	✔ Pasa
3	Simular un fallo de carga de chunk (ej. deploy con caché de chunks obsoleta)	Mensaje "No pudimos cargar esta página" y botón "Recargar página", en vez de una pantalla en blanco	✔ Pasa

4	Click en "Recargar página"	Exactamente una recarga completa del navegador por click	✔ Pasa
5	Navegar fuera de una ruta que falló y volver a ella	El estado de error se limpia automáticamente al cambiar de ruta (`resetKey`), sin recargar toda la página	✔ Pasa
6	Verificar `<html lang>` y `<title>` del documento	`lang="es"` , título fijo "TuHospedaje"	✔ Pasa
7	Usuario con `prefers-reduced-motion` activado	La animación del spinner se desactiva	✔ Pasa

Cobertura automatizada: `RouteChunkErrorBoundary.test.jsx` (3 casos), `RouteLoadingFallback.test.jsx` (4 casos, fake timers), `documentMetadata.test.jsx` .

TC-46: Shell Responsive Móvil (PR #73)

- **Historias de Usuario Asociadas:** Transversal — navegación y shell responsive móvil
- **Precondiciones:** Frontend activo. Playwright ejecutado con el proyecto `mobile-chromium` .
- **Tipos de Verificación:** Test E2E Automatizado (Playwright, Chromium móvil).

VIEWPORT	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
`390x844` y `320x844`	Cargar una ruta pública y recorrer header, contenido y footer	No existe overflow horizontal; los textos y enlaces envuelven sin solapamientos ni cortes	✔ Pasa
`390x844` y `320x844`	Inspeccionar botón de menú y enlaces táctiles del shell	Los objetivos táctiles tienen tamaño y separación utilizables	✔ Pasa
`390x844` y `320x844`	Activar, cerrar y usar el menú móvil	El menú es accesible, muestra estado abierto/cerrado, permite navegar y no deja bloqueado el contenido	✔ Pasa

TC-47: Reservas Responsive y Cancelación Móvil (PR #75)

- **Historias de Usuario Asociadas:** US #33 (Historial de reservas) y US #42 (Cancelar una reserva propia)
- **Precondiciones:** Usuario autenticado con una reserva `CONFIRMED` propia y check-in futuro. Playwright ejecutado con `mobile-chromium` .
- **Tipos de Verificación:** Test E2E Automatizado (Playwright, Chromium móvil).

VIEWPORT	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
`390x844` y `320x844`	Cargar `/my-reservations` con una reserva	La tarjeta muestra datos, acciones y wrapping sin overflow; los controles tienen touch targets utilizables	✔ Pasa
`390x844` y `320x844`	Activar "Cancelar" y confirmar la interacción	Se solicita confirmación; al aceptar, la fila actualiza su estado a `CANCELLED` y no se ofrece una acción inválida después	✔ Pasa

2. Resumen de Ejecución

TIPO DE PRUEBA	CANTIDAD	ESTADO
Tests automatizados de backend (JUnit 5 + MockMvc + Testcontainers)	Histórico: 381 tests en el cierre de Sprint 4; actual: 422 tests	✔ 422/422 pasaron en CI

Tests automatizados de frontend (Vitest + React Testing Library)	Histórico: 326 tests en 46 archivos; actual: 416 tests en 53 archivos	**✔ 416/416 pasaron en CI**
Tests E2E desktop de Playwright	Histórico: 45 escenarios × 2 navegadores; actual: Chromium 44 pasaron/1 omitido y Firefox 44 pasaron/1 omitido	✔ CI actual pasó ambos jobs desktop; cada navegador registró 1 skip
Tests E2E mobile de Playwright (`mobile-chromium`)	5 escenarios actuales: 3 shell + 2 reservas	**✔ 5/5 pasaron en CI**
Jobs de CI actuales	5: backend, frontend, desktop Chromium, desktop Firefox, mobile Chromium	**✔ 5/5 pasaron en `31435735979` sobre `cd2bdee`**
Casos de Prueba Funcionales (Plan)	152 escenarios	**✔ 152/152 verificados**

Los totales marcados como históricos corresponden al merge commit `8a3fd43` y a la auditoría posterior (`2bced9d` , `091df56` , `30caab9` y `1e11b5e`). La evidencia vigente de `main` es la ejecución `31435735979` sobre `cd2bdee` ; los E2E desktop y mobile se informan por separado.

2.1. Estado actual de verificación

En `main` en `cd2bdee76b4a6031f1ebf0cdf3539d4e30245e89` , la ejecución de CI `31435735979` pasó los cinco jobs publicados: backend, frontend, desktop Chromium E2E, desktop Firefox E2E y mobile Chromium E2E. Los jobs desktop tuvieron 44 aprobados y 1 omitido por navegador; mobile tuvo 5 aprobados en total (3 shell y 2 reservas).

La cobertura actual de sesiones renovables incluye `POST /api/auth/refresh` , `POST /api/auth/logout` y `POST /api/auth/password` , además de la emisión de refresh cookies durante login y registro. La rotación, revocación y detección de replay se ejercitan mediante cobertura de servicio e integración; no se indica aquí un nuevo total de tests porque este addendum fija el estado actual y la evidencia de CI en lugar de reescribir los conteos históricos.

3. Cobertura por Historia de Usuario

USER STORY	CANTIDAD TC	TIPO	ESTADO
US #30 — Seleccionar fecha	6 TC	Automatizado (backend + frontend) + Manual	**✔ Completo**
US #31 — Visualizar detalles	9 TC	Automatizado (frontend) + Manual	**✔ Completo**
US #32 — Realizar reserva	10 TC	Automatizado (backend + frontend) + Manual	**✔ Completo**
US #33 — Historial de reservas	9 TC	Automatizado (backend + frontend) + Manual	**✔ Completo**
US #34 — WhatsApp	5 TC	Automatizado (frontend) + Manual	**✔ Completo**
US #35 — Email de confirmación	5 TC	Automatizado (backend) + Manual	**✔ Completo**

TC-36 — Edición admin (complementaria, Inc. 1)	8 TC	Automatizado + Manual	✔ Completo
TC-37 — Suite E2E Playwright (agregado, Inc. 1)	45 TC	Automatizado E2E	✔ Completo
US #36 — Tablas administrativas uniformes (Inc. 2)	5 TC	Automatizado (frontend) + Manual	✔ Completo
US #37 — Dashboard, reservas recientes (Inc. 2)	5 TC	Automatizado (backend + frontend) + Manual	✔ Completo
US #38 — Búsqueda multi-categoría paginada (Inc. 3)	7 TC	Automatizado (backend + frontend) + Manual	✔ Completo
US #39 — Mensajes de error localizados (Inc. 3)	5 TC	Automatizado (backend)	✔ Completo
US #40 — Ruta admin con `/unauthorized` (Inc. 3)	4 TC	Automatizado (frontend) + Manual	✔ Completo
US #41 — Autenticación por cookie HttpOnly + CSRF (Inc. 4)	7 TC	Automatizado (backend + frontend) + Manual (`curl`)	✔ Completo
US #42 — Cancelación de reserva propia (Inc. 4)	10 TC	Automatizado (backend + frontend) + Manual	✔ Completo
US #43 — Carga diferida resiliente de rutas (Inc. 4)	7 TC	Automatizado (frontend) + Manual	✔ Completo

4. Herramientas Utilizadas

HERRAMIENTA	PROPÓSITO
JUnit 5 + Mockito	Tests unitarios de servicios
MockMvc + Testcontainers	Tests de integración con MariaDB efímera
Playwright	Tests E2E en Chromium y Firefox
Postman	Pruebas manuales de API
Mailtrap (SMTP sandbox)	Verificación de emails de confirmación
Navegador (Chrome)	Verificación de UI
Swagger UI	Documentación y exploración de endpoints

5. Defectos Encontrados y Corregidos

ID	DESCRIPCIÓN	SEVERIDAD	ESTADO
BUG-01	`BookingPage.jsx` prefillaba el teléfono desde `data[data.length - 1]` (la reserva más antigua), cuando la API retorna por `checkIn DESC` y la reserva más reciente es `data[0]`. El test lo describía como "latest prior reservation" pero verificaba el último elemento — inconsistencia entre semántica del test y comportamiento real.	Baja (UX)	✔ Corregido — `data[0]` ; test actualizado para reflejar el orden DESC real de la API.

BUG-02 (Inc. 3)	Al validar `page` / `size` con `@Validated` + `@Min` en `LodgingController`, Spring lanza `ConstraintViolationException` — sin un handler dedicado, el catch-all `Exception.class` preexistente la interceptaba antes que la resolución nativa de Spring, devolviendo HTTP 500 en vez de 400. Detectado empíricamente durante el desarrollo (no se asumió el comportamiento, se verificó con tests), antes de mergear.	Media (contrato de API)	✔ Corregido — handler dedicado agregado en el mismo cambio; tests de regresión (`shouldReturnBadRequestWhenSearchPageIsNegative`, `...SizeIsNotPositive`) verifican 400.
BUG-03 (Inc. 3)	`spring.messages.fallback-to-system-locale=true` (default de Spring Boot) hacía que, en un host con locale del sistema operativo en español, pedir `Accept-Language: en` (o no enviar el header) devolviera igualmente el mensaje en español — comportamiento no determinístico según el entorno de ejecución, no reproducible de la misma forma en todos los hosts/CI.	Media (i18n no determinístico)	✔ Corregido — `spring.messages.fallback-to-system-locale=false` explícito en `application.properties` (main y test).
BUG-04 (Inc. 4)	La estrategia por defecto de Spring Security (`CsrfAuthenticationStrategy`) rotaba la cookie CSRF en cada request autenticado bajo `SessionCreationPolicy.STATELESS`, no solo en el login — sin `HttpSession`, no hay dónde recordar "ya procesado". Esa rotación competía con los requests paralelos que dispara la SPA tras el login, dejando la cookie del navegador desincronizada del header leído por el frontend — causa raíz de cierres de sesión intermitentes con CSRF inválido, detectados primero de forma esporádica en Firefox.	Alta (autenticación/sesión)	✔ Corregido — root-caused leyendo el código fuente de Spring Security 6.5 y reproducido de forma determinística con `curl` antes de implementar el fix; reemplazado por `NullAuthenticatedSessionStrategy` con rotación puntual y atómica en login/registro. Una implementación intermedia (limpiar y regenerar en dos pasos) introdujo un segundo defecto relacionado (dos `Set-Cookie` para el mismo nombre, rompiendo el siguiente logout) — detectado con un test real que falló, corregido con una única escritura.
BUG-05 (Inc. 4)	Un test de integración de cancelación de reservas construía su fixture con `LocalDate.now()` del reloj del sistema en vez del reloj de negocio (`America/Argentina/Buenos_Aires`); cerca de la medianoche en Buenos Aires (UTC-3) ambos relojes podían discrepar de fecha calendario, haciendo intermitente en CI el supuesto "el check-in es hoy".	Baja (test únicamente, sin código de producción afectado)	✔ Corregido — test inyecta el mismo `Clock` de negocio que usa la aplicación.